

Lab Sheet 02

Part 01 : Fundamentals in R

1. Functions in R

What are Functions?

Functions in R are reusable blocks of code designed to perform specific tasks. They can take inputs (arguments), perform operations, and return results.

2. Tasks for User-Defined Functions

1. Write a function that prints "Hello, R!" to the console.

Function definition

```
hello_function = function() {  
  print("Hello, R!")  
}
```

Function call

```
hello_function()
```

2. Create a function that takes two numbers as input and returns their product.

Function definition

```
multiply_numbers = function(a, b) {  
  return(a * b)  
}
```

Function call

```
result = multiply_numbers(4, 5)
```

```
print(result)
```

3. Write a function that calculates the square of a number and adds 10 to the result.

```
# Function definition
```

```
calculate_square = function(x) {  
  return((x^2) + 10)  
}
```

```
# Function call
```

```
square_result = calculate_square(6)  
print(square_result)
```

3. Tasks for Anonymous Functions

4. Create an anonymous function to calculate the cube of a number and call it immediately.

```
# Anonymous function
```

```
print((function(x) x^3)(3))
```

5. Use an anonymous function with `sapply()` to double the values in a numeric vector.

```
# Numeric vector
```

```
values = c(5, 10, 15, 20)
```

```
# Using sapply with an anonymous function
```

```
result = sapply(values, function(x) x * 2)
```

```
print(result)
```

6. Apply an anonymous function to calculate the sum of squares for each row in a data frame.

```
# Data frame
```

```
data_frame = data.frame(A = c(2, 4, 6), B = c(3, 5, 7))
```

```
# Using apply with an anonymous function
```

```
row_sums = apply(data_frame, 1, function(x) sum(x^2))
```

```
print(row_sums)
```

4. Advanced Task with Functions

7. Create a function that computes the perimeter of a rectangle by calling another function to calculate its area. The perimeter function should take the length and width as inputs.

```
# Area function
```

```
calculate_area = function(length, width) {  
  return(length * width)  
}
```

```
# Perimeter function
```

```
calculate_perimeter = function(length, width) {  
  area = calculate_area(length, width)  
  return(2 * (length + width))  
}
```

```
# Function call
```

```
perimeter_result = calculate_perimeter(5, 3)
```

```
print(perimeter_result)
```

5. Libraries in R

What are Libraries?

Libraries in R are collections of pre-written functions and datasets that extend the functionality of R. Popular libraries include dplyr, ggplot2, and shiny.

Examples of Libraries

1. Data Manipulation

- Library: dplyr

2. Visualization

- Library: ggplot2

3. Machine Learning

- Library: randomForest

4. Web Applications

- Library: shiny

Additional Notes

- To install a library: `install.packages("library_name")`
- To check available datasets: `data()`
- Example: Load the built-in iris dataset and view its structure.
- `data("iris")`

- `str(iris)`

Part 02 : Statistical Summaries with R

1. Descriptive Statistical Summaries

Descriptive statistics help summarize and describe the main features of a dataset. The iris dataset contains information about the sepal length, sepal width, petal length, and petal width of different species of flowers.

Basic Summaries

Load the iris dataset

```
data("iris")
```

```
head(iris) # View the first few rows
```

Summary statistics

```
summary(iris)
```

Mean of each numerical column

```
sapply(iris[, 1:4], mean)
```

Standard deviation of each numerical column

```
sapply(iris[, 1:4], sd)
```

Range of each numerical column

```
sapply(iris[, 1:4], range)
```

2. Visualizations Using Base R

2.1 Bar Charts

Create a bar chart to display the count of each species in the dataset.

Bar chart

```
species_count = table(iris$Species)
```

```
barplot(species_count, main = "Count of Species", col = "lightblue")
```

2.2 Pie Charts

Visualize the proportion of each species using a pie chart.

Pie chart

```
pie(species_count, main = "Proportion of Species", col = rainbow(3))
```

2.3 Histograms

Create a histogram for Sepal Length to understand its distribution.

Histogram

```
hist(iris$Sepal.Length, main = "Histogram of Sepal Length", col = "lightgreen",  
xlab = "Sepal Length")
```

2.4 Box Plots

Visualize the distribution of Sepal Length using a box plot.

Box plot

```
boxplot(iris$Sepal.Length, main = "Box Plot of Sepal Length", col = "pink")
```

2.5 Multiple Bar Charts

Create a grouped bar chart to compare Sepal Length across species.

Multiple bar charts

```
sepal_mean = tapply(iris$Sepal.Length, iris$Species, mean)
barplot(sepal_mean, main = "Mean Sepal Length by Species", col = c("red",
"blue", "green"), beside = TRUE)
```

2.6 Side-by-Side Box Plots

Create side-by-side box plots for Sepal Length across species.

Side-by-side box plots

```
boxplot(Sepal.Length ~ Species, data = iris, main = "Sepal Length by Species",
col = c("red", "blue", "green"))
```

2.7 Scatter Plots

Visualize the relationship between Sepal Length and Petal Length using a scatter plot.

Scatter plot

```
plot(iris$Sepal.Length, iris$Petal.Length, main = "Scatter Plot: Sepal vs Petal
Length",
      xlab = "Sepal Length", ylab = "Petal Length", col = iris$Species, pch = 19)
```

3. Correlation

What is Correlation?

Correlation measures the strength and direction of a linear relationship between two variables. The correlation coefficient ranges from -1 to 1:

- **1**: Perfect positive correlation
- **-1**: Perfect negative correlation
- **0**: No correlation

Calculating Correlation in R

```
# Correlation between Sepal Length and Petal Length  
correlation = cor(iris$Sepal.Length, iris$Petal.Length)  
print(correlation)
```

4. Correlation Matrix

What is a Correlation Matrix?

A correlation matrix displays the correlation coefficients between multiple variables in a dataset.

Calculating Correlation Matrix

```
# Correlation matrix for numerical columns  
cor_matrix = cor(iris[, 1:4])  
print(cor_matrix)
```

Visualizing Correlation Matrix

```
# Visualize the correlation matrix  
image(1:4, 1:4, cor_matrix, axes = FALSE, main = "Correlation Matrix", col =  
heat.colors(10))  
axis(1, at = 1:4, labels = colnames(iris)[1:4])  
axis(2, at = 1:4, labels = colnames(iris)[1:4])
```